

DLDCA Lab 2: Assembly

Week 2

August 26, 2025

Introduction

In this lab we are going to start with a simple set of problems through which we will understand the basics of x86 assembly syntax and semantics.

Necessary tools

We shall be using nasm in order to compile x86 assembly code.

Before starting the lab, ensure that the tool is installed in your system. This can be verified by running the following:

```
$ nasm
nasm: fatal: no input file specified
Type nasm -h for help.
```

Structure of submission/templates

```
your-roll-no/
|- task1/
    |- call-function.asm (TODO)
    |- caller-callee.asm
    |- compile.sh
|- task2/
    |- conditional-statements.asm (TODO)
    |- compile.sh
```

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of your-roll-no.tar.gz. The command given below can be used to do the same

```
$ tar -cvzf your-roll-no.tar.gz your-roll-no/
```

Tasks

An overview of tasks can be seen in TODO.md. **Please go through this file before starting the lab.**

Task 1: More about registers

The goal of this task is learn more about caller and callee saved registers.

To get started, first run caller-callee through gdb just to refresh about what caller and callee saved registers are. In a nutshell, caller saved registers are those whose values may change across a function or system call. Thus, it becomes the responsibility of the caller to save and restore these registers before and after a function or system call.

To save registers, code writers may use the push instruction to "push" the value of the registers to the top of the stack (which is nothing but a part of your memory). To restore registers, code writers may use the pop instruction to "pop" the value on the top of the stack and store it in a register.

Your job in this task, is to modify call-function.asm so that before and after the function call to func_callee is made, all registers except rax, hold the same value.

To run your code, you may run:

```
$ bash compile.sh  
$ ./call-function
```

OR you can do it manually using,

```
nasm -f elf64 call-function.asm -o call-function.o  
ld call-function.o -o call-function  
./call-function
```

Task 2: Conditional Statements in asm

The goal of this task is to learn about how to implement conditional statements like if-else statements in x86 assembly.

You have to take a number as input from the user using the read system call (system call number is 0) with the arguments being:

1. 0 (in rdi) - to indicate input taken from stdin
2. input_buf (in rsi) - to indicate input number's string is to be stored in input_buf
3. 20 (in rdx) - Which is the maximum number of digits of the number that we will be giving

We have then given code which converts the number from being a string to being an actual 64-bit number in the rax register.

Given the number, you then have to find if it gives remainder 5 with 7 (i.e. if $number \% 7 == 5$), if it does, you have to output fizz. Similarly, if it is even, you have to output buzz. If both are true, output fizzbuzz. If neither are true, output nothing. **PLEASE NOTE THERE IS NO NEW LINE IN THE OUTPUT.** A newline is printed when you give the input but the output should not have a newline.

You can use the write syscall (syscall number = 1) to give output to the terminal. The arguments are:

1. 1 (in rdi) - to indicate writing to stdout
2. fizz_msg OR buzz_msg (in rsi) - to indicate what to write to stdout
3. 4 (in rdx) - to indicate how many bytes to write to stdout

For example, here are some expected outputs

```
$ ./conditional-statements
19
fizz$
```

```
$ ./conditional-statements
4
buzz$
```

```
$ ./conditional-statements
26
fizzbuzz$
```

```
$ ./conditional-statements
7
$
```

You are encouraged to use the assembly instructions cheatsheet which has been given to you to understand the various instructions that may be used to implement conditional statements.

To run your code, you may run:

```
$ bash compile.sh
$ ./conditional-statements
```

OR you can do it manually using,

```
nasm -f elf64 conditional-statements.asm -o conditional-statements.o
ld conditional-statements.o -o conditional-statements
./conditional-statements
```

Assembly instructions cheatsheet (non-exhaustive)

Instruction	Operands	Effect
mov	reg, reg/*mem/imm	$\text{reg} \leftarrow \text{reg} / * \text{mem} / \text{imm}$
mov	*mem, reg	$* \text{mem} \leftarrow \text{reg}$
add	reg, reg/*mem/imm	$\text{reg} \leftarrow \text{reg} + \text{reg} / * \text{mem} / \text{imm}$
add	*mem, reg/imm	$* \text{mem} \leftarrow * \text{mem} + \text{reg} / \text{imm}$
sub	reg, reg/*mem/imm	$\text{reg} \leftarrow \text{reg} - \text{reg} / * \text{mem} / \text{imm}$
sub	*mem, reg/imm	$* \text{mem} \leftarrow * \text{mem} - \text{reg} / \text{imm}$
xor	reg, reg/*mem/imm	$\text{reg} \leftarrow \text{reg} \oplus \text{reg} / * \text{mem} / \text{imm}$
xor	*mem, reg/imm	$* \text{mem} \leftarrow * \text{mem} \oplus \text{reg} / \text{imm}$
and	reg, reg/*mem/imm	$\text{reg} \leftarrow \text{reg} \wedge \text{reg} / * \text{mem} / \text{imm}$
and	*mem, reg/imm	$* \text{mem} \leftarrow * \text{mem} \wedge \text{reg} / \text{imm}$
or	reg, reg/*mem/imm	$\text{reg} \leftarrow \text{reg} \vee \text{reg} / * \text{mem} / \text{imm}$
or	*mem, reg/imm	$* \text{mem} \leftarrow * \text{mem} \vee \text{reg} / \text{imm}$
imul	reg, reg/*mem/imm	$\text{reg} \leftarrow \text{reg} \times \text{reg} / * \text{mem} / \text{imm}$
imul	*mem, reg/imm	$* \text{mem} \leftarrow * \text{mem} \times \text{reg} / \text{imm}$
div	reg	$\text{rax} \leftarrow (\text{rdx}:\text{rax}) / \text{reg}$ $\text{rdx} \leftarrow (\text{rdx}:\text{rax}) \% \text{reg}$
cmp	reg/*mem, reg/imm	Sets flags based on subtraction: $\text{reg} / * \text{mem} - \text{reg} / \text{imm}$. ZF = 1 if equal, SF = 1 if result negative, CF = 1 if borrow (unsigned), OF = 1 if signed overflow.
test	reg/*mem, reg/imm	Sets flags based on bitwise AND: $\text{reg} / * \text{mem} \wedge \text{reg} / \text{imm}$. ZF = 1 if result is zero, SF = 1 if high bit set, CF and OF are cleared. No result is stored.
inc	reg/*mem	$\text{reg} / * \text{mem} \leftarrow \text{reg} / * \text{mem} + 1$
dec	reg/*mem	$\text{reg} / * \text{mem} \leftarrow \text{reg} / * \text{mem} - 1$
jmp	label	Unconditional jump to label
je / jz	label	Jump if Zero Flag (ZF) = 1
jne / jnz	label	Jump if Zero Flag (ZF) = 0
jl	label	Jump if Less (SF $\neq$ OF)
jle	label	Jump if Less or Equal (ZF = 1 or SF $\neq$ OF)
jg	label	Jump if Greater (ZF = 0 and SF = OF)
jge	label	Jump if Greater or Equal (SF = OF)
syscall		Performs a syscall with syscall number stored in rax Arguments passed in other registers starting from rdi
push	reg	Pushes the value in reg onto the stack
pop	reg	Pops the value on top of the stack onto reg
call	label	Calls the function called label

Table 1: Common x86-64 Instructions and Their Effects